Week 4 Security Assessment Report

Application: User Management System (Express + MongoDB) **Prepared by:** Muhammad Raza **Date:** 11 March 2026

---

1. Executive Summary

This week focused on implementing and validating intrusion detection and prevention for the User Management System. The team integrated Fail2Ban with Winston logging and applied API hardening techniques to detect, log, and mitigate brute-force and unauthorized access attempts. This report details the technical implementation, evidence, and security impact, with all screenshots linked and described inline.

---

2. Methodology

- **Fail2Ban Integration:** Custom jail and filter for Node.js login attempts, monitoring Winston log output.
- **Winston Logging:** Captured all authentication and security events for audit and response.
- **API Hardening:** Applied CORS, Helmet, and rate limiting to all endpoints.
- **Validation:** Manual endpoint testing and log review.

---

3. Implementation & Evidence

3.1 Fail2Ban Jail & Filter

To detect brute-force login attempts, a custom Fail2Ban jail was created to monitor the Winston-generated log file for failed logins.

Fail2Ban jail configuration:

```
razajavaid@Kali: ~
Session  Actions  Edit  View  Help
  GNU nano 8.6                                                    /etc/fail2ban/jail.local
[devhub-node]
enabled = true
port = 8080
filter = devhub-node
# Point to your actual project log path
logpath = /home/razajavaid/Desktop/User-Management-Security-Assessment/User-Management-System-CRUD/security.log
maxretry = 5
bantime = 15m
findtime = 15m
# Ensure Fail2Ban interprets the winston ISO timestamp as UTC
logtimezone = UTC
datepattern = "timestamp":"{^LN-BEG}%%Y-%%m-%%dT%%H:%%M:%%S\.%%fZ"
# Configure alert system (sends an email when an IP is banned) [cite: 84]
destemail = your-email@example.com
sender = fail2ban@localhost
mta = sendmail
action = %(action_mwl)s
# Leave ignoreip blank to allow testing bans on localhost
ignoreip =
```

Fail2Ban filter regex:

```
razajavaid@Kali: ~/Desktop/User-Management-Security-Assessment
Session  Actions  Edit  View  Help
  GNU nano 8.6                                                    /etc/fail2ban/filter.d/node-login.conf
[Definition]
failregex = ^.*"level":"warn","message":"Invalid (login|password) attempt for email: .* from IP: (?:::ffff:)?<HOST>".*$
ignoreregex =
```

When repeated failed logins were detected, Fail2Ban automatically banned the offending IP address. The status output below shows 12 failed attempts and the currently banned IP:

```
Status for the jail: devhub-node
|- Filter
|  |- Currently failed: 1
|  `- Total failed: 12
|  |- File list:
|     /home/razajavaid/Desktop/User-Management-Security-Assessment
|-t/User-Management-System-CRUD/security.log
`- Actions
   |- Currently banned: 1
   |- Currently banned: 1
   |- Total banned: 1
   `- Banned IP list: 192.168.56.50

fail2ban.actions: WARNING [devhub-node] Ban 192.168.56.50
```

3.2 Security Logging

Winston logging was configured to capture all security events, including:

- Invalid login attempts
- Brute-force attempts blocked
- Unauthorized API access
- Successful secure API endpoint access

Security log evidence:

```
┌──(razajavaid㉿kali)-[~/Desktop/User-Management-Security-Assessment/User-Management-System-CRUD]
└─$ tail -n 15 security.log
{"level":"warn","message":"Invalid login attempt for email: test@test.com from IP: ::ffff:192.168.56.50","timestamp":"2026-03-11T03:57:44.778Z"}
{"level":"warn","message":"Brute-force attempt blocked for IP: ::ffff:192.168.56.50","timestamp":"2026-03-11T03:57:44.818Z"}
{"level":"info","message":"Login attempt for email: test@test.com","timestamp":"2026-03-11T04:04:41.924Z"}
{"level":"warn","message":"Invalid login attempt for email: test@test.com from IP: ::ffff:192.168.56.50","timestamp":"2026-03-11T04:04:41.947Z"}
{"level":"info","message":"Login attempt for email: test@test.com","timestamp":"2026-03-11T04:04:42.009Z"}
{"level":"warn","message":"Invalid login attempt for email: test@test.com from IP: ::ffff:192.168.56.50","timestamp":"2026-03-11T04:04:42.016Z"}
{"level":"info","message":"Login attempt for email: test@test.com","timestamp":"2026-03-11T04:04:42.085Z"}
{"level":"warn","message":"Invalid login attempt for email: test@test.com from IP: ::ffff:192.168.56.50","timestamp":"2026-03-11T04:04:42.096Z"}
{"level":"info","message":"Login attempt for email: test@test.com","timestamp":"2026-03-11T04:04:42.142Z"}
{"level":"warn","message":"Invalid login attempt for email: test@test.com from IP: ::ffff:192.168.56.50","timestamp":"2026-03-11T04:04:42.147Z"}
{"level":"info","message":"Login attempt for email: test@test.com","timestamp":"2026-03-11T04:04:42.184Z"}
{"level":"warn","message":"Invalid login attempt for email: test@test.com from IP: ::ffff:192.168.56.50","timestamp":"2026-03-11T04:04:42.191Z"}
{"level":"warn","message":"Brute-force attempt blocked for IP: ::ffff:192.168.56.50","timestamp":"2026-03-11T04:04:42.232Z"}
{"level":"warn","message":"Unauthorized API access attempt from IP: ::ffff:192.168.56.50","timestamp":"2026-03-11T04:11:24.527Z"}
{"level":"info","message":"Secure API endpoint accessed successfully","timestamp":"2026-03-11T04:11:41.936Z"}
```

3.3 API Hardening

The following security measures were implemented:

- **CORS restrictions** to limit allowed origins
- **Helmet middleware** for security headers
- **Rate limiting** to block brute-force attempts and log them for Fail2Ban

API hardening code:

```
const rateLimit = require('express-rate-limit');
const cors = require('cors');

// 1. Properly configure CORS to restrict unauthorized access
router.use(cors({
  // UPDATED: Include both localhost and your network IP for testing
  origin: ['http://localhost:8080', 'http://192.168.56.50:8080'],
  optionsSuccessStatus: 200
}));

// 2. Implement security headers with proper configuration

router.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"],
      objectSrc: ["'none'"],
      upgradeInsecureRequests: [],
    },
  },
  hsts: {
    maxAge: 31536000, // 1 year
    includeSubDomains: true,
    preload: true
  }
}));

// 3. Apply rate limiting to prevent brute-force attacks
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5,
  message: 'Too many login attempts, please try again later.',
  handler: (req, res, next, options) => {
      // Logic: Logs the block so Fail2Ban can catch the "Brute-force attempt blocked" string
      logger.warn(`Brute-force attempt blocked for IP: ${req.ip}`);
      res.status(options.statusCode).send(options.message);
  }
});
```

---

4. Endpoint Testing & Results

Manual testing was performed to validate protections:

- **Incorrect login endpoint access** returned `404 Not Found`:

```
┌──(razajavaid㉿Kali)-[~]
└─$ curl -I http://localhost:8080/login
HTTP/1.1 404 Not Found
Vary: Origin
Content-Security-Policy: default-src 'none'
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Content-Type: text/html; charset=utf-8
Content-Length: 145
Date: Wed, 11 Mar 2026 04:09:57 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

- **Unauthorized API access** to `/api/secure-data` returned `401 Unauthorized`:

```
┌──(razajavaid㉿Kali)-[~]
└─$ curl -i http://192.168.56.50:8080/api/secure-data
HTTP/1.1 401 Unauthorized
Vary: Origin
Content-Security-Policy: default-src 'self';script-src 'self';object-src 'none';upgrade-insecure-requests;base-uri 'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'self';img-src 'self' data:;script-src-attr 'none
';style-src 'self' https: 'unsafe-inline'
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Content-Type: text/html; charset=utf-8
Content-Length: 31
ETag: W/"1f-K335WIhVjHMIuGLAiWf2H2ctEfI"
Date: Wed, 11 Mar 2026 04:11:24 GMT
Connection: keep-alive
Keep-Alive: timeout=5

Access denied. Invalid API key.
```

- **Brute-force attempts** triggered rate limiting and Fail2Ban ban:

```
┌──(razajavaid㉿Kali)-[~]
└─$ for i in {1..6}; do curl -X POST http://192.168.56.50:8080/login -H "Content-Type: application/json" -d '{"email":"test@test.com", "password":"wrongpassword"}'; echo ""; done
Invalid credentials
Invalid credentials
Invalid credentials
Invalid credentials
Invalid credentials
Too many login attempts, please try again later.
```

---

5. Impact & Security Analysis

- **Real-time detection and response:** Fail2Ban and Winston logging provided immediate visibility and automated blocking of brute-force attacks.
- **Audit trail:** Security logs offer a clear record of all suspicious and legitimate activity.
- **API resilience:** CORS, Helmet, and rate limiting significantly reduced the attack surface and mitigated common web threats.

---

6. References

- Fail2Ban Documentation
- Winston Logger
- Helmet.js Security
- OWASP API Security Top 10

---

7. Conclusion

By the end of Week 4:

- Fail2Ban and Winston logging were integrated for real-time detection and response.
- Custom jail and filter successfully detected and banned brute-force attempts.
- Security logs provided clear evidence of attacks and mitigations.
- API endpoints were hardened with CORS, Helmet, and rate limiting.

This demonstrates a complete defensive workflow: **Detect → Log → Ban → Harden**